Strategy Design Pattern

Table of contents

STRATEGY DESIGN PATTERNS

> "Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it"

[GoF, "Design Patterns: Elements of Reusable Object-Oriented Software"]

Let us consider the `fizzbuzz` exercise, as well as any function whose behavior depends on a parameter (switch) that controls a complex conditional structure.

```
void fizzBuzz(int i, int div1, int div2, int sw_f){
  Bool fizz, buzz;
  switch (sw_f){
    case 1:
      fizz= isMultiple(i, div1);
      buzz= isMultiple(i, div2);
      break;
    case 2:
      fizz= isGreaterThan(i, div1);
      buzz= isGreaterThan(i, div2);
      break;
  }
//...
}

int main(){
  int i, div1, div2, sw_f;

  i = 10;
  div1=3;
  div2=5;
  sw_f=1; //switch value: 1 or 2

  fizzBuzz( i,  div1,  div2, sw_f);
}
```

Problems in this implementation

1. Conditional logic

- The function may contain complex, nested conditional logic with k branches.
- This increases maintenance challenges, making the code harder to understand and modify.
- Every time the function's behavior changes, the conditional structure must be updated — adding new branches, modifying conditions, and increasing the risk of introducing bugs.

2.Coupling

- Coupling defines the level of interdependence between modules. High coupling occurs when two modules are strictly dependent on each other, whereas low (or loose) coupling allows each module to be analyzed, understood, modified, tested, or reused independently.
- The worst form of coupling is **control coupling**, where the client module passes a flag (a switch, in this case) that dictates the server module's behavior and internal logic.
- This violates encapsulation because the client (`main`) becomes aware of the server's (`fizzbuzz` function) internal details. If `fizzbuzz` changes its implementation — for example, if `sw_f` starts from 0 instead of 1, or if it becomes a string — the client's code breaks and requires modification.

---

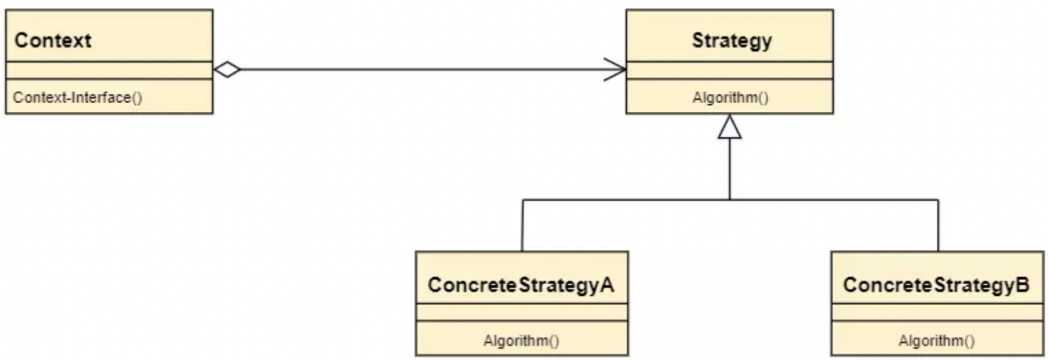
3. Scalability Issues

Due to the design flaws mentioned above, this solution lacks scalability. If new behavior types need to be added, the inflexible design forces modifications to the function itself. Changing existing code is often error-prone, increasing the risk of introducing new bugs.

Strategy design pattern

Design pattern STRATEGY provides a way to follow and reap the benefits of the open-closed principle.

The Strategy Pattern is a behavioral design pattern that allows **selecting an algorithm at runtime**. Instead of implementing a single algorithm directly, the code is designed to receive runtime instructions on which algorithm - from a defined family of algorithms - to execute.

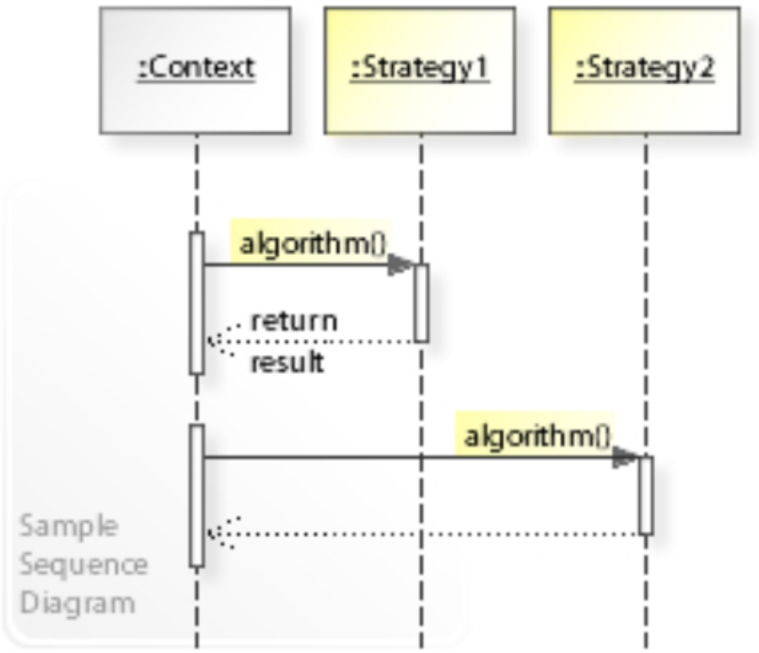


In the `UML class diagram` above, the `Context` class[1] does not implement the algorithm directly.

Instead, it delegates the algorithm's execution to an object that adheres to the `Strategy` interface by calling `strategy.algorithm()`.

This design decouples `Context` from specific algorithm implementations, promoting flexibility and maintainability.

The `ConcreteStrategyA` and `ConcreteStrategyB` classes provide different algorithm implementations of the algorithm, which `Context` can **select and invoke dynamically at runtime**.



The `sequence diagram` above illustrates the message flow between elements of the software system. The `Context` object requests an operation, but the specific algorithm used (the **concrete strategy**) is determined at runtime.

The `context` interacts only with the generic `strategy` interface, remaining unaware of the actual implementation. The same `algorithm()` call dinamically invokes either `ConcreteStrategyA` or `ConcreteStrategyB`, depending on runtime conditions.

[1] If you're unfamiliar with object-oriented programming (*OOP*), you can think of 'class' as a 'module' for now.

Strategy design pattern - C implementation

> **Note**
>
> The discussion about the strategy design pattern and the problems analyzed above are entirely general andand independent of any specific programming language.
> The implementation shown below, which uses function pointers, is specific to the C programming language. However, the Strategy pattern itself is a language-independent design approach.

Memory stores both instructions and data. Just as we can define a pointer to data (e.g., `int *p`), we can also define a pointer to a function.

A function pointer is a variable that holds the address of a function.

Since it is a *pointer*, it can be dereferenced, assigned to another pointer, and manipulated like any other pointer.

Ponter-to-function Syntax

```c
// function
int sum(int a, int b);

// pointer to a function
int (*p_sum)(int a, int b);


// Assign the function address to the pointer
p_sum = sum; // Equivalent to p_sum = &sum;

// dereference the pointer and invoke the function
s = p_sum(c1,c2);  // Equivalent to s = (*p_sum)(c1,c2);
```

Both syntaxes (`p_sum = sum; s = p_sum(c1,c2)` and `p_sum = &sum; s = (*p_sum)(c1,c2)`) are equivalent. By convention, the shorter form is generally preferred.

Using the function pointer or the function name itself has the same effect.

How to use the pointer to a function to solve the `fizzbuzz` problem?

- define a `fizzbuzz` function that accepts a function pointer as a parameter (1)
- define one or more functions that evaluate conditions for `fizzbuzz` (e.g., `isGreaterThan`, `isMultiple`, etc.). These serves as **concrete strategies** (2)
- the function pointer should reference (*point to*) one of these condition functions (3)
- all condition functions must share the same inteface: they should accept two integers as parameters and retun a boolean value.

```
// (1) fizzBuzz function
void fizzBuzz(int i, int div1, int div2,
          Bool (*p_cond)(int a, int b)){

  fizz= p_cond(i, div1);
  buzz= p_cond(i, div2);

  // ...
}


// (2) concrete strategies

Bool isMultiple(int n, int d){
    return  n%d==0;
}
// main

  // (3) assign the function address to the pointer
  p = isMultiple;
  // call the fizzbuzz with the concrete strategy
  fizzBuzz( i,  div1,  div2, p);
```

Solution - full code

The core idea of the Strategy Pattern is to **separate the general algorithm from the specific rule**, and this principle is respected here. However, the example is somewhat weak. See the next examples.

```
// FIZZBUZZ_5

/*
*
* È un'implementazione accettabile dell'idea alla base dello Strategy Pattern,
```

```
* perché usa un puntatore a funzione per rendere sostituibile una parte del
* comportamento a runtime.
* Tuttavia, come esempio di Strategy Pattern è piuttosto minimale
* e ricorda più una callback o un predicato parametrico che una strategia completa.
*
*/

#include <stdio.h>
typedef int Bool;

void fizzBuzz(int i, int div1, int div2,
          Bool (*p_cond)(int a, int b)){
  Bool fizz, buzz;
  fizz= p_cond(i, div1);
  buzz= p_cond(i, div2);

  if (fizz && buzz )
    printf("fizzbuzz\n");
  else if (fizz)
    printf("fizz\n");
  else if (buzz)
    printf("buzz\n");
  else
    printf("%d\n",i);
}

//----

Bool isMultiple(int n, int d){
    return  n%d==0;
}
Bool isGreaterThan(int n, int d){
    return  n>d;
}


int main(){
  int N=20, i, div1, div2;

  /* pointer to a function
   * it can point to isMultiple(),
   * to isGreaterThan(),
   * or to any function that accepts two integers
   * and returns an integer (Bool)
   */
```

```
  Bool (*p_condition)(int a, int b);
  p_condition= isMultiple; // oppure isGreaterThan


  // ------------------------
  div1=3;
  div2=5;
  printf("\n\n--------------\n\n");

  for (i=1; i<=N; i++){
    fizzBuzz( i,  div1,  div2, p_condition);
  }
}
```

Benefits of the open-closed principle.

- New behavior is implemented by adding new code (such as new functions or modules) rather than modifying existing code.
- Remove conditional logic from the `fizzbuzz` function.
- Decouple the `fizzbuzz` function from its clients.

EXAMPLE - customer

Implement a program that categorizes users into predefined customer types and applies different strategies for price calculation and shipping costs.

Customers fall into one of three categories:

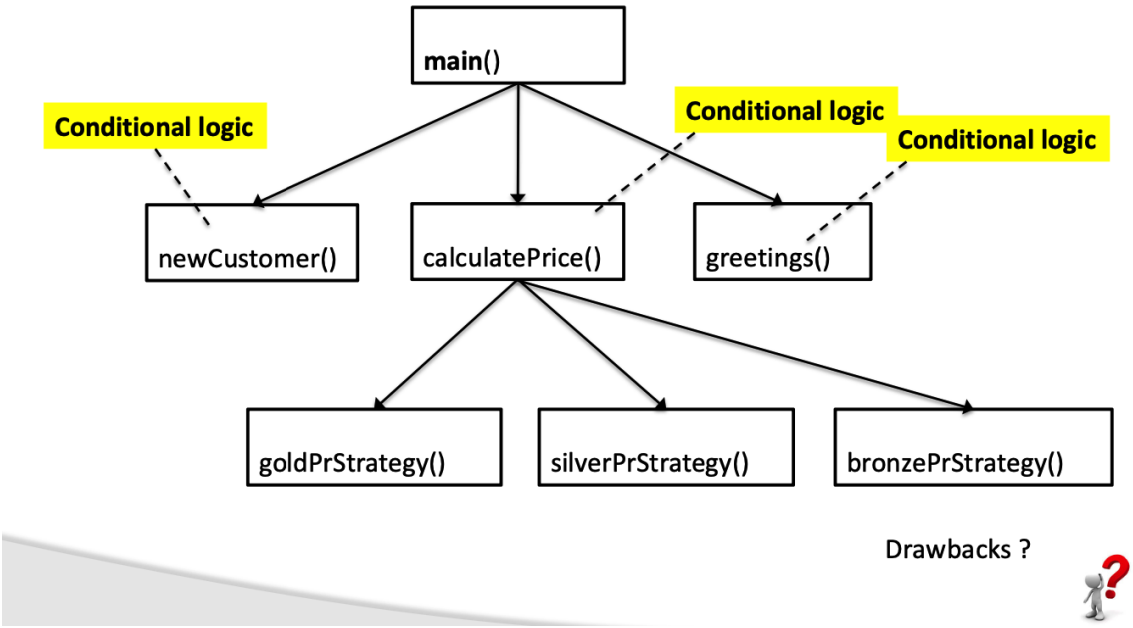
- **Bronze customers**: pay full price for every item, plus shipping costs.
- **Silver customers**: receive a 10% discount on every item, plus shipping costs.
- **Gold customers**: receive a 20% discount on every item and free shipping.

Additionally, each customer receives a greeting message on their birthday.
Add a `greetings()` function to handle this behavior.

The `greetings()` function should:

- Send a gift card to **Silver customers** (€10) and **Gold customers** (€20).
- Send only a greeting message (no gift card) to **Bronze customers**.

It is sufficient to simulate these operations using print statements.



Without the Strategy pattern, the solution must handle k customer categories and n possible actions. This results in **n conditional structures**, each containing **k branches**, leading to a complex and hard-to-maintain codebase.

A possible solution (no strategy design pattern):

```
// customer_01 main.c

/* * * * * * * * * * * * * * * * * * * * * * * * * * * * *
 * Ask for customer's name, type (gold, silver, bronze)
 * (and, optionally, for other data)
 *
 * Apply a discount that depends on the type of customer
 *
 * * * * * * * * * * * * * * * * * * * * * * * * * * * * */


/*
 * Questa implementazione contiene molti difetti
 * - mischia fgets, getchar, scanf
 * - gestione fragile dell'input
 * - ...
 * ma qui ci stiamo focalizzando su altri concetti,
```

```
 * accettiamo queste imprecisioni
 *
 */
#include <stdio.h>
#include "customerModule.h"


int main() {
    T_customer c;
    float price, finalPrice;
    const float shipping = 10;


    printf("** INSERT CUSTOMER **\n");
    newCustomer(&c);

    printf("INSERT PRICE:\n");
    scanf("%f", &price);
    printf("COMPUTE FINAL PRICE (discount, shipping, etc):\n");

    finalPrice = computePrice(c, price, shipping);
    //finalPrice=computePrice(p_c->typeOfCustomer,price, shipping);

    // OUTPUT

    printf("customer:\n");
    printf("%s -> %s customer \n", c.name, c.typeOfCustomer);

    printf("Initial price: %.2f\n", price);
    printf("Final price: %.2f\n", finalPrice);

    greetings(c);

    return 0;
}
```

```
// customer_01 customerModule.h

#ifndef CUSTOMERMODULE_H
#define CUSTOMERMODULE_H

#define DIM 50
```

```
typedef struct Customer {
    char name[DIM]; //
    char *typeOfCustomer; // gold, silver, bronze, etc
    // pessima scelta, solo per uso didattico
    // (mostrare 'che si può fare' e quali sono i limiti)
    // ma meglio usare enum o int

    // to implement:
    //      List orders; address; telephone number; etc
    //      ...
} T_customer;


void newCustomer(T_customer *p_c);

float computePrice(T_customer c, float totalAmount, float shipping);
void greetings(T_customer c);
// passare la struttura a computePrice e greetings
// è una scelta inefficiente - meglio passare il puntatore -
// ma qui voglio mettere il focus su altri aspetti

#endif //CUSTOMERMODULE_H
```

```
// customer_01 customerModule.c

#include "customerModule.h"
#include <stdio.h>
#include <string.h> // strcmp()
#include <stdlib.h> // exit()


// 'private' functions

float goldPriceStrategy(float amount, float shipping);

float silverPriceStrategy(float amount, float shipping);

float bronzePriceStrategy(float amount, float shipping);

// implementation

void newCustomer(T_customer *p_c) {

    printf("Insert customer's name: ");
```

```
    fgets(p_c->name, DIM, stdin);

    printf("Choose customer type:\n");
    printf("a: GOLD\n");
    printf("b: SILVER\n");
    printf("c: BRONZE\n");

    switch (getchar()) {
        case 'a':
            p_c->typeOfCustomer = "gold";
            // equivalent to (*p_c).typeOfCustomer=...
            break;
        case 'b':
            p_c->typeOfCustomer = "silver";
            break;
        case 'c':
            p_c->typeOfCustomer = "bronze";
            break;
        default:
            perror("Unknown customer type");
            exit(1);
    }

}


float computePrice(T_customer c, float totalAmount, float shipping) {
    /* Calculate the total price
     * depending on customer category. */
    float price;


    if (strcmp(c.typeOfCustomer, "gold") == 0) {
        price = goldPriceStrategy(totalAmount, shipping);

    } else if (strcmp(c.typeOfCustomer, "silver") == 0) {

        price = silverPriceStrategy(totalAmount, shipping);

    } else if (strcmp(c.typeOfCustomer, "bronze") == 0) {
        price = bronzePriceStrategy(totalAmount, shipping);
    } else {
        perror("Unknown customer type");
        exit(1);
    }
```

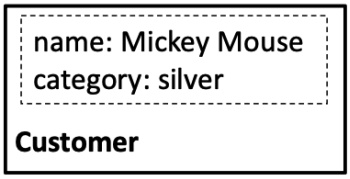
```
    return price;
}


void greetings(T_customer c) {
    /* Happy Birthday (and gift card for some categories) */
    if (strcmp(c.typeOfCustomer, "bronze") == 0) {
        printf("Happy Birthday!");
    } else if (strcmp(c.typeOfCustomer, "silver") == 0) {
        printf("Happy Birthday + gift card 10 euros!");
    } else if (strcmp(c.typeOfCustomer, "gold") == 0) {
        printf("Happy Birthday + gift card 20 euros!");
    } else {
        perror("Unknown customer type");
        exit(1);
    }
}



// PRIVATE

float goldPriceStrategy(float amount, float shipping) {
    float priceDiscount = 0.8;
    return priceDiscount * amount; // free shipping;
}


float silverPriceStrategy(float amount, float shipping) {
    float priceDiscount = 0.9;
    return priceDiscount * amount + shipping;
}

float bronzePriceStrategy(float amount, float shipping) {

    return amount + shipping;
}
```

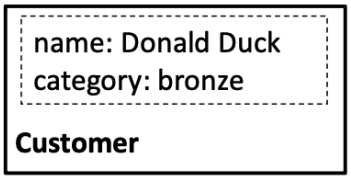
We have already discussed the limitations of this approach. With regard to scalability,

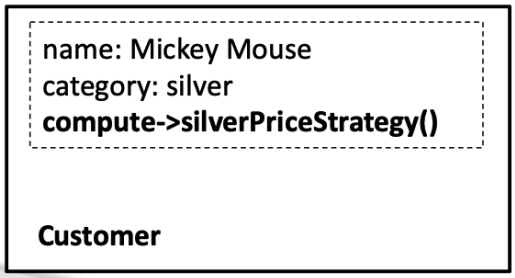
- What happens if new customer categories are introduced?
- What if additional behaviors need to follow a similar logic?

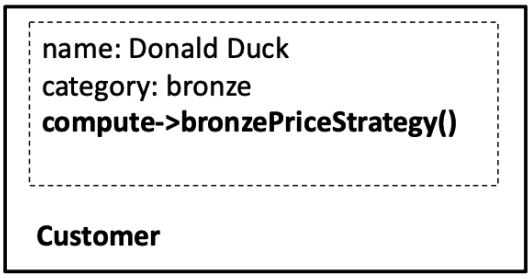
- Modifying existing code is often error-prone and can easily introduce bugs.

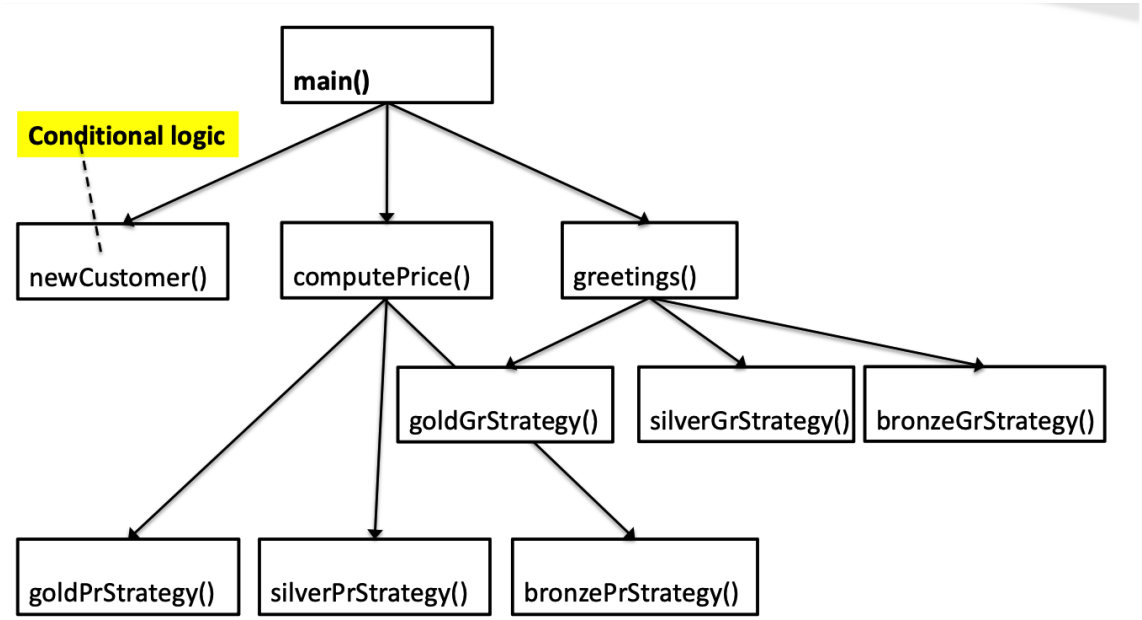
A possible solution is to incorporate customer-specific behavior directly into the data structure.











A possible solution (with strategy design pattern):

- main

```
// customer_02 main.c

/* * * * * * * * * * * * * * * * * * * * * * * * * * * *
 * Ask for customer's name, type (gold, silver, bronze)
 * (and, optionally, for other data)
 *
 * Apply a discount that depends on the type of customer
 *
 * * * * * * * * * * * * * * * * * * * * * * * * * * * */

#include <stdio.h>
#include "customerModule.h"


int main() {
    T_customer c;
    float price, finalPrice;
    const float shipping = 10;


    printf("** INSERT CUSTOMER **\n");
    newCustomer(&c);

    printf("INSERT PRICE:\n");
    scanf("%f", &price);
    printf("COMPUTE FINAL PRICE (discount, shipping, etc):\n");

    finalPrice = computePrice(c, price, shipping);

    // OUTPUT

    printf("customer:\n");
    printf("%s -> %s customer \n", c.name, c.typeOfCustomer);

    printf("Initial price: %.2f\n", price);
    printf("Final price: %.2f\n", finalPrice);

    greetings(c);
```

```
    getchar();
    return 0;
}
```

- customerModule.h

```
// customer_02 customerModule.h

#ifndef CUSTOMERMODULE_H
#define CUSTOMERMODULE_H

#define DIM 50


typedef struct Customer {
    char name[DIM]; //
    char *typeOfCustomer; // gold, silver, bronze, etc

    /* Bind the strategy to Customer using pointers to functions */
    float (*p_priceStrategy)(float amount, float shipping);

    void (*p_greetingsStrategy)();


    // to implement:
    //      List orders; address; telephone number; etc
    //      ...
} T_customer;


void newCustomer(T_customer *p_c);

float computePrice(T_customer c, float totalAmount, float shipping);

void greetings(T_customer c);

#endif //CUSTOMERMODULE_H
```

- customerModule.c

```
// customer_02 customerModule.c
```

```c
#include "customerModule.h"
#include <stdio.h>
#include <string.h> // strcmp()
#include <stdlib.h> // exit()


// 'private' functions

float goldPriceStrategy(float amount, float shipping);

float silverPriceStrategy(float amount, float shipping);

float bronzePriceStrategy(float amount, float shipping);

void goldGreetingsStrategy();

void silverGreetingsStrategy();

void bronzeGreetingsStrategy();



// implementation



//*********************************************************
//   CREATE CUSTOMER
//*********************************************************


void newCustomer(T_customer *p_c) {

    printf("Insert customer's name: ");
    fgets(p_c->name, DIM, stdin);

    printf("Choose customer type:\n");
    printf("a: GOLD\n");
    printf("b: SILVER\n");
    printf("c: BRONZE\n");

    switch (getchar()) {
        case 'a':
            p_c->typeOfCustomer = "gold";
            p_c->p_priceStrategy = goldPriceStrategy;
```

```
            p_c->p_greetingsStrategy = goldGreetingsStrategy;
            break;
        case 'b':
            p_c->typeOfCustomer = "silver";
            p_c->p_priceStrategy = silverPriceStrategy;
            p_c->p_greetingsStrategy = silverGreetingsStrategy;
            break;
        case 'c':
            p_c->typeOfCustomer = "bronze";
            p_c->p_priceStrategy = bronzePriceStrategy;
            p_c->p_greetingsStrategy = bronzeGreetingsStrategy;
            break;
        default:
            perror("Unknown customer type");
            exit(1);
    }

}



//******************************************************
//   PRICE STRATEGY
//******************************************************


float computePrice(T_customer c, float totalAmount, float shipping) {
    /* compute the total price
     * depending on customer category. */
    float price;

    price = (c.p_priceStrategy)(totalAmount, shipping);
    /* it will be
    * bronzePriceStrategy, silverPriceStrategy, goldPriceStrategy
    * or others
    * depending on the Customer*/
    return price;
}

// PRIVATE

float goldPriceStrategy(float amount, float shipping) {
    float priceDiscount = 0.8;
    return priceDiscount * amount; // free shipping;
}
```

```
float silverPriceStrategy(float amount, float shipping) {
    float priceDiscount = 0.9;
    return priceDiscount * amount + shipping;
}

float bronzePriceStrategy(float amount, float shipping) {

    return amount + shipping;
}


//*****************************************************
//   GREETINGS STRATEGY
//*****************************************************
void greetings(T_customer c) {
    // Happy Birthday (and gift card for some categories)
    (c.p_greetingsStrategy)();
}

// PRIVATE

void goldGreetingsStrategy() {
    printf("Happy Birthday + gift card 20 euros!");
};

void silverGreetingsStrategy() {
    printf("Happy Birthday + gift card 10 euros!");
};

void bronzeGreetingsStrategy() {
    printf("Happy Birthday!");
};
```

The benefits of the Open-Closed Principle include:

- New behavior can be added by introducing new code, rather than modifying existing code.
- It reduces complex conditional logic, as decisions are localized within a single module.

- It enables multiple versions of the same algorithm.
- By using function pointers, strategies can be changed at runtime simply by pointing to a different function.

Minimal example

```c
#include <stdio.h>

/* Tipi delle strategie */
typedef void (*StrategyA)(void);
typedef void (*StrategyB)(void);

/* Contesto */
typedef struct {
    StrategyA actionA;
    StrategyB actionB;
} Context;

/* Strategie concrete per A */
void A1(void) { printf("Eseguo A con strategia A1\n"); }
void A2(void) { printf("Eseguo A con strategia A2\n"); }

/* Strategie concrete per B */
void B1(void) { printf("Eseguo B con strategia B1\n"); }
void B2(void) { printf("Eseguo B con strategia B2\n"); }

/* Impostazione delle strategie */
void setStrategies(Context *c, StrategyA sa, StrategyB sb) {
    c->actionA = sa;
    c->actionB = sb;
}

/* Metodi del contesto */
void doA(Context *c) {
    if (c->actionA != NULL)
        c->actionA();
}

void doB(Context *c) {
    if (c->actionB != NULL)
        c->actionB();
}
```

```c
int main(void) {
    Context c = { NULL, NULL };

    setStrategies(&c, A1, B2);
    printf("Prima configurazione:\n");
    doA(&c);
    doB(&c);

    setStrategies(&c, A2, B1);
    printf("\nSeconda configurazione:\n");
    doA(&c);
    doB(&c);

    return 0;
}
```

The context encapsulates function pointers that represent the strategies. The concrete strategies are functions with the same signature and can be replaced at runtime without modifying the context.

The context does not know how the actions are implemented; it only knows that it can invoke them through function pointers.

Exercise

Write a program that manages two types of customers: **NORMAL** and **GOLD**. The program should implement three actions:

- `computePrice`
- `getWaitingTime`
- `getShipmentTime`

Use a data structure to store the essential customer information, including:

- name
- customer category
- function pointers for strategy-based actions

Implement a `newCustomer()` function to handle customer creation.

The program should: * Prompt the user for the customer's details. * Create the necessary data structures. * Ask how many items the customer purchased and the unit price. * Use the appropriate strategy functions to calculate the **final price**, **help desk waiting time**, and **shipping time**.

There is no need to store a list of customers or purchased products; the program only needs to process the given input.

Example data structures:

```
typedef struct{
  // function pointers for strategy-based actions
  // computePrice
  // getWaitingTime
  // getShipmentTime
}
CategoryStruct;

typedef struct  {
        char name[DIM]; //
        int type_of_customer;
        CategoryStruct category; // Stores function pointers for strategies

} Customer;
```

EXAMPLE - e-commerce

There are three activity periods:

- **Normal period**
- **Gray Friday**
- **Black Friday**

and two actions:

- **Purchase**
- **Registration on the site**

Action: Purchase

- **Normal period**: pay full price for each item, plus shipping costs.
- **Gray Friday**: receive a 10% discount on each item, plus shipping costs.
- **Black Friday**: receive a 20% discount on each item, with free shipping.

Action: Registration

- **Normal period**: requires a small fee.

- **Gray Friday**: free registration plus one voucher.
- **Black Friday**: free registration plus one maxi voucher.

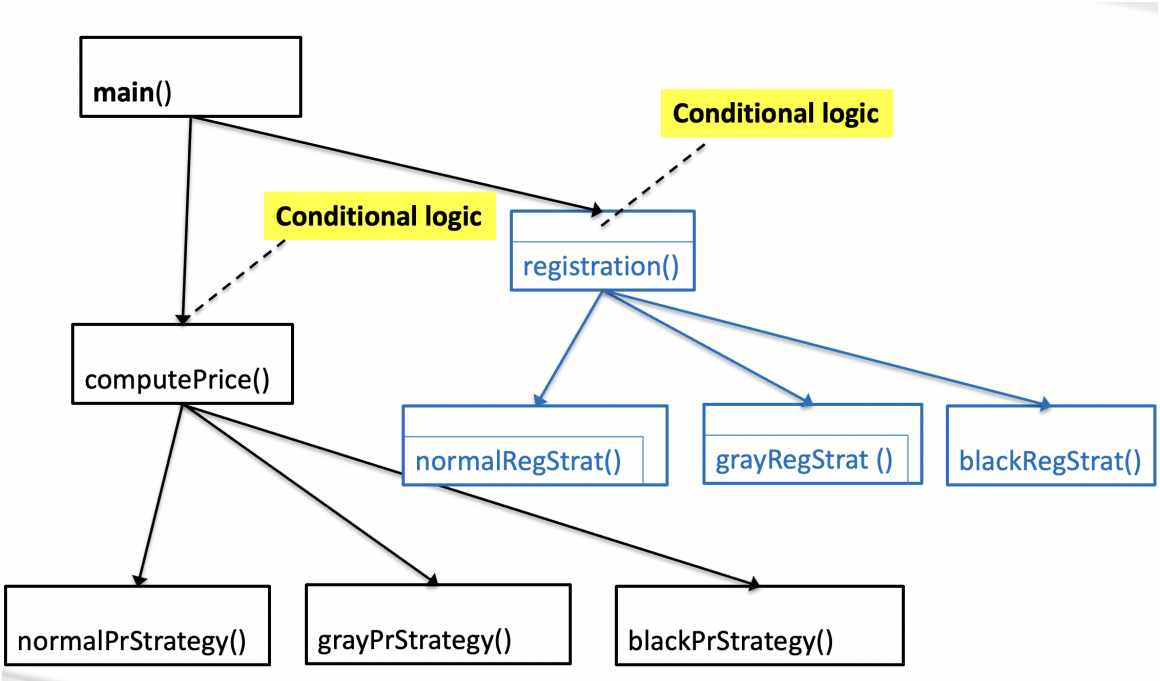
Write a program that:

- Calculates the final price based on the sales period, the number of purchased items, and their prices.
- For a new registration, determines the applicable fee and vouchers according to the sales period.

Solution (draft) without strategy design pattern

```
float computePrice(int n_item, float unit_cost, int sales_period){
  switch (sales_period){
  case 1:...
      // ...
  break;

  case 2:...
      // ...
  break;

  }

}

float computeRegistrationFee(int sales_period){
  switch (sales_period){
  case 1:...
      // ...
  break;

  case 2:...
      // ...
  break;

  }

}


float computeVoucher(int sales_period){
```

```
  switch (sales_period){
  case 1:...
      // ...
  break;

  case 2:...
      // ...
  break;

  }

}

// ----
// MAIN
// ----


sales_period = 2;

val_voucher = computeVoucher(sales_period);
total_price = computePrice( 10, 30, sales_period );
```



What happens if we introduce a new type of action or a new sales period?

We could end up with k branches in a switch-case structure, with each switch-case being duplicated across n different modules or functions.

Solution (draft) with strategy design pattern (1)

```
float normalPriceStrategy(int n_item, float unit_cost){
  return n_item * unit_cost + shipping_costs;
}

float blackPriceStrategy(int n_item, float unit_cost){
  return n_item * unit_cost *0.8;
}


float normalRegistrationFeeStrategy(){
  return smallValue;
}


// -----

float computePrice(int n_item, int unit_cost,
        float (p_price)(int n_item, float unit_cost) ){

  total_price = p_price (n_item, int unit_cost);
  return total_price;

}

// ----
// MAIN -
// ----


p_price = blackPriceStrategy;
total_price = computePrice( 10, 30, p_price );


p_registration = blackregistrationStrategy;
total_price = computeRegistration( 10, 30, p_registration );
```

Solution (draft) with strategy design pattern (2)

```
// Define function pointer types
typedef float (*PriceFunction)(int n_item, float unit_cost);
typedef float (*RegistrationFunction)(void);

// Define the struct using typedef
typedef struct {
    PriceFunction p_price;
    RegistrationFunction p_registration;
    char description[100];
    int number;
} SalesStruct;



// ----
// MAIN -
// ----


int main() {

    SalesStruct normal_sales, black_sales;

    normal_sales.p_price = normalPriceStrategy;
    normal_sales.p_registration = normalRegistrationFeeStrategy;
    normal_sales.number = ...;


    black_sales.p_price = blackPriceStrategy;
    black_sales.p_registration = blackRegistrationFeeStrategy;
    black_sales.number = ...;



    // val_voucher = computeVoucher(sales_period);
    // total_price = computePrice( 10, 30, sales_period );
```

```
    computeALL (..., black_sales)
```

Solution with strategy design pattern

The developer of the `strategy` library provides `module.c` and `module.h`, which implement the **normal** behavior (i.e., the actions performed during the *normal* period).

- `module.h`

```
// salesPeriod module.h


#ifndef MODULE_H
#define MODULE_H
#include <string.h>
// Define function pointer types
typedef float (*PriceFunction)(int n_item, float unit_cost);
typedef float (*RegistrationFunction)(void);

// Define the struct using typedef
typedef struct {
    PriceFunction p_price;
    RegistrationFunction p_registration;
    char description[100];
    int number;
} SalesStruct;

SalesStruct createNormalStrategy();
float normalPriceStrategy(int n_item, float unit_cost);
float normalRegistrationFeeStrategy();


#endif
```

- `module.c`

```
// salesPeriod module.h


#include "module.h"
```

```c
float normalPriceStrategy(int n_item, float unit_cost){
    float shipping_costs = 10;
    return n_item * unit_cost + shipping_costs;
}

float normalRegistrationFeeStrategy(){
    float fee_value = 5;
    return fee_value;
}

SalesStruct createNormalStrategy(){
    SalesStruct s;
    s.p_price = normalPriceStrategy;
    s.p_registration = normalRegistrationFeeStrategy;
    s.number=0;
    strcpy(s.description, "Normal Strategy");
    return s;
}
```

The `createNormalStrategy()` function is responsable for creating an object (a `struct` variable containing function pointers) that encapsulates all the required behaviors

---

The `main` function can then utilize these functions.

- `main`

```c
// salesPeriod main.c


#include <stdio.h>
#include "module.h"
#include "new_strategies.h"
int main() {
    float registration_price, total_price;
    SalesStruct actual_strategy;

    actual_strategy = createNormalStrategy();
    //actual_strategy = createBlackStrategy();

    registration_price = actual_strategy.p_registration();
    total_price = actual_strategy.p_price(10, 20);
```

```
    printf("I am using %s\n", actual_strategy.description);
    printf("Total price =%.2f, registration = %.2f", total_price, registration_price );
}
```

If the developer of the `main` module requires additional strategies — such as a set of strategies for the *Black Friday* period — he can create a new module (e.g., `new_strategies`) that contains all the necessary functions, including one to create a `SalesStruct` instance.

- `new_strategies.h`

```
// salesPeriod new_strategies.h


#ifndef NEW_STRATEGIES_H
#define NEW_STRATEGIES_H
#include "module.h"

/* While some guidelines suggest including header files only in .c files,
 * it's  important to ensure that every header file is self-contained.
 * new_strategy.h  should include module.h
 * because it needs SalesStruct
 *
 * */
float blackPriceStrategy(int n_item, float unit_cost);
float blackRegistrationFeeStrategy();

SalesStruct createBlackStrategy();
#endif //NEW_STRATEGIES_H
```

- `new_strategies.c`

```
// salesPeriod new_strategies.c

#include "new_strategies.h"
#include "module.h"
float blackPriceStrategy(int n_item, float unit_cost){
    return n_item * unit_cost *0.8;
}
float blackRegistrationFeeStrategy(){
    float fee_value = 0;
    return fee_value;
}
```

```
SalesStruct createBlackStrategy(){
    SalesStruct s;
    s.p_price = blackPriceStrategy;
    s.p_registration = blackRegistrationFeeStrategy;
    s.number=1;
    strcpy(s.description, "Black Friday Strategy");
    return s;
}
```

Note that

- the `main` must include both `module.h` and `new_strategies.h` headers
- `new_strategies.c` file must include the `module.h` header
- Even though some guidelines suggest including headers only in `.c` files, `new_strategies.h` must also include `module.h`because it requires the definition of `SalesStruct`
- The developer of the `main` module can add new behavior by adding new entities without modifying the existing code, adhering to the **open-closed principle**.

Exercise - MAX

Consider a function `max()` that finds the maximum element in an array of integers. The core of the algorithm is as follows:

```
Element max(v){
      for...
        if (v[i]> temp_max)
            v_max = v[i]
```

However, if you need to sort an **array of structures** with fields like `x1` and `x2`, you must specify which field (or combination of fields) should be used to determine the order. For example:

```
Element max(v){
      for...
        if (switch ==1)
          if (v[i].x1> v_max.x1)...
```

This approach clearly illustrates the limitations of **control coupling** and its inherent **lack of flexibility**, as discussed earlier.

Solution 1 (Not Satisfactory)

The idea is to write a function that extracts the desired field:

```
get_x1(Element el):
  return el.x1
```

Then, define a generic `max()` function that uses a field-extraction function pointer:

```
Element max( v, ... (*p_extract)(Element)) {

    for ... {
         actual_val = p_extract(v[i]);
         max_val = p_extract(v_max);
        if (actual_val > max_val) {
            v_max = v[i];
            ...
```

And you would call it like this:

```
p = get_x1;

result = max(v, p);
```

This approach is unsatisfactory for two main reasons:

- **Type Uniformity**: All possible `get_...` functions must return the same type (e.g., `int`), or at least types that allow acceptable implicit conversions (e.g., `int` to `float`). This limits the flexibility of the design.
- **Operator Applicability**: The returned type must support the `>` operator. This requirement is not met for types like strings or other complex structures. For example, using `>` between two strings (i.e., `char *`) compares their memory addresses rather than their actual content, which is not meaningful for ordering.

This solution, while illustrating a way to extract fields dynamically, suffers from significant limitations in both type flexibility and the ability to perform meaningful comparisons on non-primitive data types.

Solution 2 (Correct)

In this approach the strategy function we implement is exactly the comparison criterion, tailored to the fields we want to compare. For example:

```
bool compare_x1(Element a, Element  b):
  return a.x1 > b.x1

bool compare_salary(Element a, Element  b):
  return a.hours * a.euros_for_hours> b.hours * b.euros_for_hours>
```

Then, define a generic `max()` function that uses a comparison function pointer:

```
// Define
Element max(v, p_compare){
      for...


        if (p_compare (v[i], max_val))...
```

Advantages of this approach

- Flexibility: the comparison strategy is fully encapsulated in the strategy function, allowing you to define complex or composite comparisons (such as salary) without being limited to a single data type.
- Type Independence: there is no requirement for all possible extractor functions to return the same type or for the type to directly support the > operator.
- Generality: you can easily implement comparison strategies for complex data types (like strings or structures) by defining an appropriate comparison function.

This strategy-based solution overcomes the limitations of the previous approach by focusing on the comparison logic rather than on extracting and comparing specific fields.

EXERCISES

ESERCITAZIONE

Implementare una funzione che dati 3 numeri interi ne sceglie uno. Il criterio di scelta dipende dalla strategia concreta.

Definiamo due strategie:

1. `greater_than(a, b) return a>b;`
2. `less_than(a, b) return a<b;`

Poi il cliente, dal `main`, sceglierà la strategia con un pointer-to-function.

Draft soluzione

```c
#include <stdio.h>

typedef int point;

int trova_intero(int x, int y, int z, point (*p_cond)(int a, int b)) {
    point cond1, cond2, cond3;
    cond1 = p_cond(x, y);
    cond2 = p_cond(x, z);
    cond3 = p_cond(y, z);

    if (cond1 && cond2)
        return x;
    else if (!cond1 && cond3)
        return y;
    else if (!cond2 && !cond3)
        return z;
}

point greater_than(int a, int b) {  // greater than
    return a > b;
}

point less_than(int a, int b) {  // less than
    return a < b;
}



int main() {
    int x = 10, y = 20, z = 15;
    point (*p_condition)(int a, int b);
    p_condition = greater_than;

    int result = trova_intero(x, y, z, greater_than);
    printf("Il maggiore tra %d, %d e %d è: %d\n", x, y, z, result);
```

```
    p_condition = less_than;
    result = trova_intero(x, y, z, p_condition);
    printf("Il minore tra %d, %d e %d è: %d\n", x, y, z, result);

    return 0;
}
```

ESERCITAZIONE

Implementare una funzione `max_s()` che, dato un vettore di strutture `V`, sceglie la struttura con il valore più alto.

Variante 1: la strategia concreta può essere del tipo

```
get_campo_a (el){}
  return el.a;
  }
```

e la funzione `max_s` usa al suo interno

`if get_campo_a (V[i]) >get_campo_a (V[i+1]) ...`

Variante 2:

Suggerimento: L'idea è che il client definisce la struttura, i suoi campi e anche le strategie concrete che definiscono il criterio d'ordine. Allora, meglio fare una funzione che accetti sempre due strutture e renda sempre un booleano.

```
compare_campo_a (e1, el2){
  return el1.a> el2.a;
  }
```

qui se i campi non sono direttamente confrontabili - es. strighe, vettori ecc - chi scrive la strategy è responsabile del confronto

---

DRAFT SOLUZIONE

```c
// Variante 1 (semplice ma più debole)

#include <stdio.h>
#include <string.h>

// struct che può essere passata tramite interfaccia.h (header)
// oppure definita dal client
typedef struct data {
    int   età;
    float peso;
    char  nome[20];
    char  animale[20];
} Elemento;

// funzione per estrarre il campo età
int get_campo_età(Elemento e) {
    return e.età;
}

// funz per estrarre campo peso (con cast a int, scelta discutibile)

// nota: meglio fare sempre cast a float piuttosto che int per non perdere informazioni
int get_campo_peso(Elemento e) {
    return (int)e.peso;  // attenzione: può causare problemi
}


// funz per estrarre campo nome (impossibile fare max, con cast ancora peggio)
int get_campo_nome(Elemento e) {
    return e.nome;
}

// max_s basata su get_campo  - variante 1
Elemento max_s_variante1(Elemento* v, int n, int (*get_campo)(Elemento)) {
    int max_idx = 0;
    int cond;
    for (int i = 1; i < n; i++) {
        cond= get_campo(v[i]) > get_campo(v[max_idx]);
        if (cond) {
            max_idx = i;
        }
    }
    return v[max_idx];
}
```

```c
// MAIN VARIANTE 1
int main() {

    // esempio: "database" di un veterinario (forse non molto bravo perchè ha solo 5 pazie
    Elemento v[] = {
        {5,  4.6,   "Londra",   "Gatto"},
        {7,  4.8,   "Leonardo", "Gatto"},
        {8,  35.2,  "Kira",     "Cane"},
        {2,  1.5,   "Arturo",   "Coniglio"},
        {16, 690.5, "Mafalda",  "Mucca"}
    };

    int n = sizeof(v) / sizeof(v[0]);

    Elemento max = max_s_variante1(v, n, get_campo_età);
    printf("Max by campo età: %d %f %s %s\n", max.età, max.peso, max.nome, max.animale);

    max = max_s_variante1(v, n, get_campo_peso);  // qui faccio cast da float a int!
    printf("Max by campo peso (int cast): %d %f %s %s\n", max.età, max.peso, max.nome, max

    return 0;
}
```

```c
////////////////////////////////////////////////////////////////////////////////////////////
// Variante 2 (serve attenzione con le strategy ma è flessibile, generalizzabile, potente)

#include <stdbool.h>

// struct uguale alla variante 1
typedef struct data {
    int   età;
    float peso;
    char  nome[20];
    char  animale[20];
} Elemento;


// ho usato BOOL come tipo di dato per le funzioni e per il point-to-funct
// posso usare anche int, l'importante è che sia coerente il tipo di dato
```

```
// che restituiscono le funzione e il tipo di dato del point-to-funct

// strategy 1: confronto sul campo età (intero)
bool compare_by_età(Elemento e1, Elemento e2) {
    return e1.eta > e2.eta;
}

// strategy 2: confronto sul campo peso (float)
bool compare_by_peso(Elemento e1, Elemento e2) {
    return e1.peso > e2.peso;
}

// strategy 3: confronto sul nome
// confronto ordine alfabetico? uso strcmp
// confronto lunghezza stringa? uso strlen
bool compare_by_nome(Elemento e1, Elemento e2) {
    return strcmp(e1.nome, e2.nome) < 0; // -1 è true o false? true!
    //return strlen(e1.nome) > strlen(e2.nome);
}


// max_s - variante dos
Elemento max_s_variante2(Elemento* v, int n, bool (*compare)(Elemento, Elemento)) {
    int max_idx = 0;
    for (int i = 1; i < n; i++) {
        if (compare(v[i], v[max_idx])) {  // ho direttamente un bool dentro il controllo d
            max_idx = i;
        }
    }
    return v[max_idx];
}

// MAIN VARIANTE 2
int main() {

    // esempio: "database" di un veterinario (forse non molto bravo perchè ha solo 5 pazie
    Elemento v[] = {
        {15,  4.6,  "Londra",   "Gatto"},
        {7,  4.8,   "Leonardo", "Gatto"},
        {8,  35.2,  "Kira",     "Cane"},
        {2,  1.5,   "Arturo",   "Coniglio"},
        {12, 690.5, "Mafalda",  "Mucca"}
    };

    int n = sizeof(v) / sizeof(v[0]);
```

```
    Elemento max = max_s_variante2(v, n, compare_by_età);
    printf("Max by campo età: %d %.2f %s %s\n", max.età, max.peso, max.nome, max.animale);

    max = max_s_variante2(v, n, compare_by_peso);
    printf("Max by campo peso: %d %.2f %s %s\n", max.età, max.peso, max.nome, max.animale)

    max = max_s_variante2(v, n, compare_by_nome);
    printf("Max by campo nome: %d %.2f %s %s\n", max.età, max.peso, max.nome, max.animale)

    return 0;
}
```

ESERCITAZIONE

In un sito e-commerce sono presenti due azioni principali: `vendita()` e `voucher_omaggio()`. In base al periodo (es. giorni normali, Black Friday), cambiano le strategie da applicare.

Implementare una struttura che rappresenti una strategia composta da due puntatori a funzione:

```
typedef struct {
void (*vendita)(void);
void (*voucher)(void);
} StrategySet;
```

Implementare almeno due versioni concrete per ogni funzione (es. `vendita_normal_strategy()`, `vendita_black_strategy()`).

Il client, dal `main()`, sceglierà la strategia da applicare in base a una variabile (es. `black_friday = 1`) e verranno eseguite le funzioni corrispondenti

---

DRAF SOLUZIONE

```
#include <stdio.h>

// DICHIARAZIONE STRUTTURA E STRATEGIE (FUORI DAL MAIN, NEL MODULO.C)

// typedef generici strategy funct
```

```c
typedef void (*vendita_ptf)(void);
typedef void (*voucher_ptf)(void);

typedef struct {
    vendita_ptf vendita;
    voucher_ptf voucher;
} StrategySet;

// strategie normali
void vendita_normal(void) {
    printf("Vendita a prezzo pieno :(\n");
}

void voucher_normal(void) {
    printf("Nessun voucher disponibile :(\n");
}

// strategie per black friday
void vendita_black_friday(void) {
    printf("Vendita con sconto per il Black Friday! :)\n");
}

void voucher_black_friday(void) {
    printf("Voucher regalo attivato! :)\n");
}

// altre funzioni utili: acquisto, sconti, rimborsi, iscrizioni


// METODI DI INIZIALIZZAZIONE DEI PROFILI - normal e black_friday (qui sono solo due, ma p
// N.B. LE INIZIALIZZAZIONI DELLE STRUCT VANNO INSERITE NEL MAIN!
    // 1. opzione "rapida"
    StrategySet normal = { vendita_normal, voucher_normal };
    StrategySet black_friday = { vendita_black_friday, voucher_black_friday };


    // 2. opzione classic way
    StrategySet normal;
    normal.vendita = vendita_normal;
    normal.voucher = voucher_normal;

    StrategySet black_friday;
    black_friday.vendita = vendita_black_friday;
    black_friday.voucher = voucher_black_friday;
```

```
    // 3. funzione per init: scrivo una funzione dedidcata (nel modulo.c) per inizializzar
    void init_strategies(StrategySet* normal, StrategySet* black_friday) {
        normal->vendita = vendita_normal;
        normal->voucher = voucher_normal;
        black_friday->vendita = vendita_black_friday;
        black_friday->voucher = voucher_black_friday;
    }
    init_strategies(&normal, &black_friday); // QUESTA LA CHIAMO NEL MAIN!

    // 4. altra opzione esotica
    StrategySet normal = {
        .vendita = vendita_normal,
        .voucher = voucher_normal
    };

    StrategySet black_friday = {
        .vendita = vendita_black_friday,
        .voucher = voucher_black_friday
    };



// MAIN CON SOLUZIONE 1: PUNTATORE A STRUTTURA CHE PUNTA ALLA STRUTTURA CORRETTA IN BASE A

// open cross priciple -> altre strategie con stessa struct di partenza
int main() {

    // puntatore a struttura per scegliere quale profilo/strategy eseguire (normal o black
    StrategySet* strategy;

    // init dei profili normal e black_friday -> classic way
    StrategySet normal;
    normal.vendita = vendita_normal;
    normal.voucher = voucher_normal;

    StrategySet black_friday;
    black_friday.vendita = vendita_black_friday;
    black_friday.voucher = voucher_black_friday;

    // siamo in periodo Black Friday!
    int scenario = 1; // 0: normal, 1: black friday

    // logica per scegliere la strategy
    if (scenario) {         // black friday !!
        strategy = &black_friday;   // punto al profilo black friday (quindi ho accesso ai
```

```
    } else {                // normal strategy
        strategy = &normal;     // punto al profilo normal (ho accesso ai campi della stru
    }

    // e poi applico la strategia, ovvero eseguo le funzioni
    strategy->vendita();
    strategy->voucher();

    return 0;
}



// MAIN CON SOLUZIONE 2: UNA SOLA STRUCT "generic" INIZIALIZZATA DIRETTAMENTE DENTRO L'IF-

int main() {

    // struttura generica che verrà inizializzata dentro l'if-else
    StrategySet generic;

    int scenario = 1; // 0: normal, 1: black friday

    // logica per scegliere la strategy
    if (scenario) {                         // black friday !!
        generic.vendita = vendita_black_friday; // init della struct generic - caso black_
        generic.voucher = voucher_black_friday;
    } else {                    // normal strategy
        generic.vendita = vendita_normal;   // init della struct generic - caso normal
        generic.voucher = voucher_normal;
    }

    // e poi applico la strategia, ovvero eseguo le funzioni
    generic.vendita();
    generic.voucher();

    return 0;
}
```

EXERCISE

- **A Simple Calculator with Pluggable Operations**

Goal: Build a basic calculator that can perform operations like addition, subtraction, multiplication, and division. Use the strategy pattern via function pointers to switch between these operations at runtime.

EXERCISE

- Character Behavior in a Game

Goal: Create a character that can switch between different attack strategies — like using a sword, bow, or magic. You'll use function pointers to change behavior dynamically at runtime.

Variants:

- Add defend strategies
- Let the character have a name.
- Create multiple characters with different default strategies.
- Simulate a battle system where characters switch strategies mid-fight.

EXERCISE

- Sort

Write a program that can sort an array of integers using different algorithms. Use function pointers to dynamically choose which sorting strategy to use. If you only know one sorting algorithm, use function pointers to dynamically choose whether to sort in ascending or descending order.

EXERCISE

- Logger System with Pluggable Output Strategies

Build a logger that can write messages using different strategies: to the console, to a file, or to a buffer in memory. You'll use function pointers to switch between output targets dynamically.

EXERCISE

- AI Movement in a Grid Game

Goal: Simulate a simple grid game where AI enemies have different movement strategies — for example:

- Aggressive: moves toward the player
- Defensive: moves away from the player
- Random: picks a random direction

Use function pointers to swap out the enemy's movement behavior dynamically.

Variants:

- Multiple enemies with different behaviors
- Switch strategies mid-game (e.g. if health $< 50\%$, switch to defensive)

NOTE

One might assume that the Strategy design pattern is just a workaround to simulate an object-oriented approach in non-OOP languages or a way to avoid inheritance in OOP ones. While it's true that we could achieve similar behavior by creating a separate class for each scenario (or more precisely, an abstract class with multiple subclasses representing different cases), the Strategy pattern offers distinct advantages beyond that.

Consider a system with n different pricing strategies and m independent registration strategies. Without the Strategy pattern, we would need to implement m $\times$ n distinct classes to cover every possible combination—leading to an unwieldy, hard-to-maintain codebase. Instead, by encapsulating each strategy separately, we can dynamically select and combine them at runtime, resulting in a more flexible and manageable design.

References

[Tornhill 2015] Adam Tornhill. "Patterns in C." https://leanpub.com/patternsinc

[Martin 2002] Martin. "Agile Software Development: Principles, Patterns, and Practices"

[Gamma 1994] Gamma et al., "Design Patterns: Elements of Reusable Object-Oriented Software"